

## บทที่ 2

### ทฤษฎีที่เกี่ยวข้อง

ในบทนี้จะว่าด้วยเรื่องของทฤษฎีต่าง ๆ ที่เกี่ยวข้องกับโครงงานนี้ ซึ่งแบ่งออกได้เป็น 2 ส่วน คือ ทฤษฎีทางด้านการประมวลผลภาพ และ ทฤษฎีทางด้านของ Neural network ซึ่งมีเนื้อหาดังต่อไปนี้

#### 2.1 ทฤษฎีทางด้านการประมวลผลภาพ

หลังจากที่เรา scan รูปเข้ามาแล้ว ขั้นตอนที่เราจะทำต่อไปก็คือการ ทำภาพให้อยู่ในรูปของ grayscale เพื่อที่จะเอารูปนั้นมาทำการ threshold อีกทีหนึ่ง สำหรับขั้นตอนของการทำภาพจาก rgb ให้เป็น grayscale นั้นสามารถทำได้ดังขั้นตอนต่อไปนี้

เนื่องจากว่า model สีแบบ rgb นั้นประกอบด้วย 3 channel คือ r ขนาด 8 bit , g ขนาด 8 bit และ b ขนาด 8 bit เช่นกัน ดังนั้นจึงเก็บโดยใช้ขนาด 24 bit หรือ 3 byte ต่อ 1 pixel เมื่อทำการ convert มาเป็น grayscale จะใช้เพียงแค่ 1 byte ในการเก็บแต่ละ pixel

$$Y = 0.3 * R + 0.59 * G + 0.11 * B$$

R คือ component ของสีแดง

G คือ component ของสีเขียว

B คือ component ของสีน้ำเงิน

Y คือค่าของ grayscale

หลังจากที่เราเอาภาพมาทำเป็น grayscale ขั้นตอนต่อไปที่เราทำก็คือการเอาภาพที่เป็น grayscale มาทำการ threshold เพื่อทำภาพให้อยู่ในรูปของ binary หรือว่าภาพที่มีสองสีนั่นเอง(ได้แก่สีขาวกับสีดำ) ที่เราต้องเอาภาพมาทำการ binarization ก็เพื่อที่จะเอาภาพนั้นมาทำการหาบริเวณของตัวอักษร สำหรับการ threshold นั้นมีสอง คือ

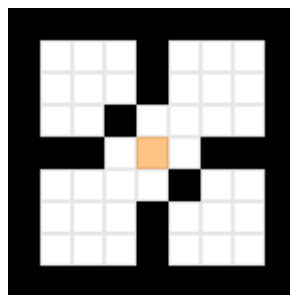
##### 1. การ threshold แบบธรรมดา

การ threshold แบบนี้เรียกได้อีกอย่างหนึ่งว่า global threshold เนื่องจากว่าเราจะเลือกค่าที่จะเอามาทำการ threshold กับภาพเพียงแค่ค่าเดียวทั้งนั้น และ apply กับทุก ๆ ค่า pixel ในภาพเลย

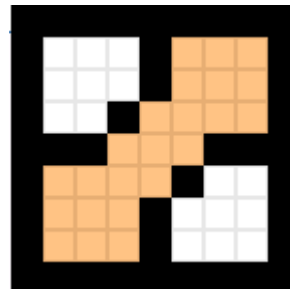
##### 2. การ threshold แบบ adaptive

วิธีการที่ตรงตามชื่อของมันคือ การ threshold แบบนี้จะเลือกค่าที่จะเอามาทำการ threshold ได้ หลาย ๆ ค่า โดยเราก็จะ apply ค่าที่เลือกได้นี้กับบริเวณใดบริเวณหนึ่งที่อยู่ในภาพเท่านั้น ไม่ใช้กับทุก ๆ pixel ในภาพหรอก ส่วนใหญ่บริเวณที่เราเลือก apply นั้นมักจะเป็นบริเวณที่เป็นขนาดสีเหลี่ยมที่ได้มีการกำหนดไว้จากผู้เขียนโปรแกรมเอง

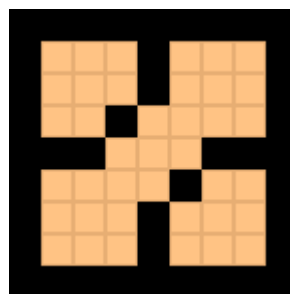
เมื่อได้ภาพที่เป็นแบบ binary แล้ว(ซึ่งก็คือภาพที่เกิดจากกระบวนการ threshold) ขึ้น ตอนต่อไปที่เราจะทำก็คือเราจะเอาภาพแบบ binary ข้างต้นมาหา blob (blob คือกลุ่มของ pixel ที่อยู่ใน ภาพที่เราสนใจที่จะเอามาทำอะไรซักอย่างหนึ่ง ในที่นี้ซึ่งเป็นการทำ OCR blob ก็จะหมายถึงตัวอักษร ที่เป็นภาพ ซึ่งจะแสดงเป็นสีขาว ส่วนตัวที่เราไม่สนใจ หรือว่าไม่ใช่ตัวอักษรนั้นก็จะถูกแสดงด้วยสีดำ) ในการหา blob นั้นเราจะต้องทำการแยก blob ทั้งหมดออกมาจากภาพให้ได้ ซึ่งวิธีการที่เขาใช้กันก็มีอยู่ ได้หลายวิธี แต่วิธีที่จะนำเสนอในที่นี้เรียกว่าการ flood fill ซึ่ง flood fill นั้นหมายถึงการเติมให้เต็ม แต่ เราสามารถใช้ flood fill เพื่อที่จะบ่งบอกถึงขอบเขตของ blob รวมทั้งที่ตั้งของ blob นั้น ๆ ได้ด้วยการ flood fill สามารถ แสดงให้เห็นได้ดังนี้



รูปที่ 2.1 ภาพแสดงตำแหน่งเริ่มต้นของการ flood fill



รูปที่ 2.2 การทำ flood fill แบบ 4 ทิศทาง



รูปที่ 2.3 การทำ flood fill แบบ 8 ทิศทาง

ในที่นี้ได้เลือกใช้วิธีการ flood fill แบบ 8 ทิศทางเนื่องจากจะทำให้ผลลัพธ์ที่ดีกว่าและแสดงถึงการเชื่อมต่อได้ดีกว่า แบบ 4 ทิศทาง นอกจากนี้การ implement อัลกอริทึมของการ flood fill นั้นได้ใช้ stack มาช่วยด้วย เพราะเห็นว่าการเรียกใช้ อัลกอริทึม flood fill แบบ recursive นั้นอาจจะทำให้ stack เต็มได้

การ flood fill ใน project นี้ ได้ใช้การ flood fill จากสีขาวซึ่งเป็นสีของ blob ที่เราสนใจ เราจะ flood บริเวณที่เป็นสีขาวให้เป็นสีดำ เนื่องจากจะเป็นการตัด blob ที่เราได้ทำการ flood fill เสร็จแล้วทิ้งไป จะได้ไม่ต้องมา flood fill ซ้ำหลายรอบ

สำหรับ code คร่าว ๆ ของ algorithm การ flood fill นั้นแสดงได้ดังนี้

```
void floodFill8Stack(int x, int y, int newColor, int oldColor)
{
    if(newColor == oldColor) return; //avoid infinite loop
    emptyStack();

    if(!push(x, y)) return;
    while(pop(x, y))
    {
        screenBuffer[x][y] = newColor;
        if(x + 1 < w && screenBuffer[x + 1][y] == oldColor)
        {
            if(!push(x + 1, y)) return;
        }
        if(x - 1 >= 0 && screenBuffer[x - 1][y] == oldColor)
        {
            if(!push(x - 1, y)) return;
        }
        if(y + 1 < h && screenBuffer[x][y + 1] == oldColor)
        {
            if(!push(x, y + 1)) return;
        }
        if(y - 1 >= 0 && screenBuffer[x][y - 1] == oldColor)
        {
```

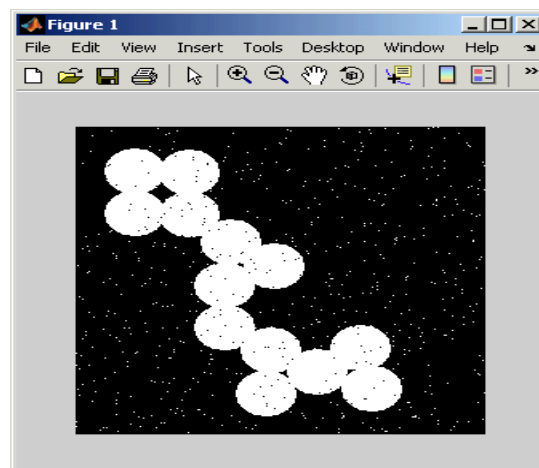
```

        if(!push(x, y - 1)) return;
    }
    if(x + 1 < w && y + 1 < h && screenBuffer[x + 1][y + 1] == oldColor)
    {
        if(!push(x + 1, y + 1)) return;
    }
    if(x + 1 < w && y - 1 >= 0 && screenBuffer[x + 1][y - 1] == oldColor)
    {
        if(!push(x + 1, y - 1)) return;
    }
    if(x - 1 >= 0 && y + 1 < h && screenBuffer[x - 1][y + 1] == oldColor)
    {
        if(!push(x - 1, y + 1)) return;
    }
    if(x - 1 >= 0 && y - 1 >= 0 && screenBuffer[x - 1][y - 1] == oldColor)
    {
        if(!push(x - 1, y - 1)) return;
    }
}
}

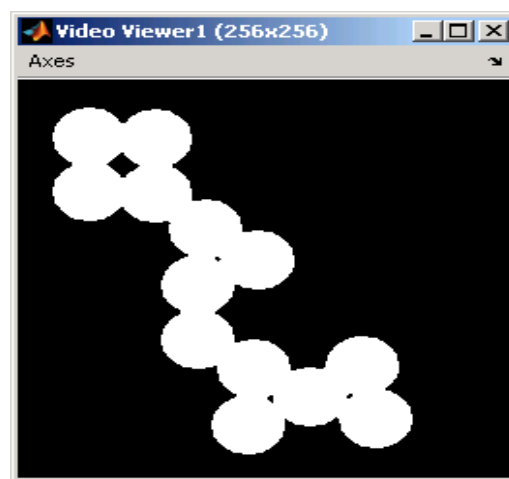
```

### Noise reduction ใน OCR

สำหรับภาพที่เราได้รับเข้ามานั้นบางทีแล้วอาจจะมี noise ปะปนเข้ามากับภาพก็ได้ เนื่องจากว่าเราต้องรับภาพจากเครื่อง scan เพราะฉะนั้น noise ชนิดหนึ่งที่จะเกิดขึ้นได้ก็คือ salt and pepper noise ซึ่ง noise ชนิดนี้ ถ้าหากว่าเราไม่ทำการกำจัดมันออกไปก่อนที่จะเอารูปนั้นไปเข้าสู่กระบวนการทางด้าน recognition อื่น ๆ ก็จะทำให้กระบวนการทางด้าน image (recognition) เกิดปัญหาขึ้นได้ อย่างเช่นทำให้เราได้สิ่งที่เราไม่ต้องการแทนที่เราจะได้รูปของตัวอักษรจริง ๆ เป็น ต้น salt and pepper noise เมื่อเกิดขึ้นกับรูปใด ๆ แล้ว ก็จะทำให้รูปนั้นมีลักษณะคล้าย ๆ กับรูปดังแสดงข้างล่างนี้



รูปที่ 2.4 ภาพแบบไบนารีที่มี noise ปนอยู่



รูปที่ 2.5 ภาพแบบไบนารีที่ผ่านการกำจัด noise แล้ว

จากรูปจะเห็นว่า salt and pepper noise นั้นทำให้เกิดจุดบริเวณเล็ก ๆ โดด ๆ โดยจะกระจายทั่ว ๆ ไปในบริเวณของรูปนั้น ๆ ถ้าเราต้องการที่จะกำจัด salt and pepper noise เราก็จะต้องลบจุดสีดำที่เรียกกันว่า pepper ทิ้งไป และนอกจากนี้ salt and pepper noise ยังทำให้เกิดหลุมในรูปได้เช่นกัน ซึ่งเราเรียกหลุมนี้กันว่า salt

หนึ่งในวิธีการที่ใช้ในการกำจัด salt and pepper noise แบบนี้ก็คือ การใช้ median filtering แต่ว่าสำหรับใน project ที่ทำอยู่ในขณะนี้ จะใช้วิธีการที่คล้าย ๆ กันก็คือ จะทำการกำหนด window ขนาด 3 X 3 ซึ่งเราจะเอา window นี้ไปวางไว้ตรงจุดที่เราสนใจโดยให้จุดตรงกลางของ window ทับกันกับจุดที่เราสนใจ จากนั้นเราก็จะนับจำนวนของจุดสีขาวทั้งหมดที่อยู่ภายใน window นี้ แล้วก็ทำการเปรียบเทียบว่าถ้าจำนวนของจุดสีขาวที่นับได้นั้นมีค่าน้อยกว่า 2 ก็จะกำหนดจุดที่เราสนใจให้กลายเป็นสีดำ แต่ถ้าหากว่าจุดสี

ขาวที่นับได้นั้นมีค่ามากกว่า 5 ก็จะกำหนดจุดที่เราสนใจให้กลายเป็นสีขาว ถ้านอกเหนือจากนั้นก็จะกำหนดให้จุดที่เราสนใจ ไม่มีการเปลี่ยนแปลงคือเป็นสีอะไรมาก่อนก็เป็นสีนั้น ๆ อยู่อย่างเดิม

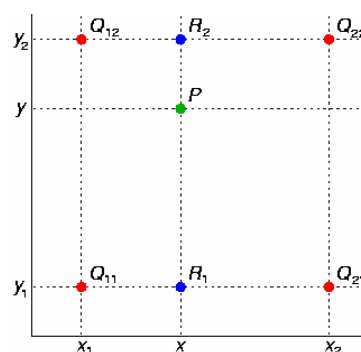
### การ Interpolation

เนื่องจากว่าเมื่อเราสามารถที่จะแยก blob ของตัวอักษรออกมาได้หมดแล้ว แต่ว่า blob ที่แยกได้นั้น อาจจะมีขนาดไม่เท่ากันกับขนาดของ blob ซึ่งใช้ในการ train ที่เอาไว้ใช้ในการ recognition ดังนั้นเพื่อเป็นการแก้ปัญหาสำหรับภาพที่รับเข้ามา ที่มีขนาดของตัวอักษรต่าง ๆ กันได้ เราเลยจำเป็นต้องทำการ scale blob ให้อยู่ในขนาดที่เป็น มาตรฐาน ขนาดใดขนาดหนึ่งซะก่อน ถ้าหากว่า blob ใด ๆ มีขนาดเล็กกว่าขนาดมาตรฐาน ก็จำเป็นต้องขยายขนาดให้ใหญ่ขึ้น และถ้าหากว่า blob มีขนาดที่ใหญ่กว่าขนาดมาตรฐาน ก็จำเป็นต้องลดขนาดให้เล็กลงด้วย กระบวนการย่อหรือขยายนี้เราเรียกว่าเป็น การ interpolation (หรือเรียกว่าการ scaling ก็ได้เช่นกัน) เทคนิคที่ใช้ในการ interpolate ในปัจจุบันมีอยู่หลายเทคนิคมากมาย อาทิเช่น การใช้วิธี interpolate แบบ nearest neighbor , bilinear interpolation , bicubic interpolation เป็นต้น แต่ในโครงงานนี้ วิธีที่ผู้จัดทำได้ใช้ คือได้เลือกใช้วิธีการของ bilinear interpolation ซึ่งถือว่าเป็นวิธีที่มีความซับซ้อนไม่มากนัก และง่ายต่อการ implement ด้วย

ต่อไปจะพูดถึงขั้นตอนวิธีในการทำ bilinear interpolation ซึ่งมีขั้นตอนดังต่อไปนี้

ในทางคณิตศาสตร์ bilinear interpolation คือส่วนเพิ่มเติมของ linear interpolation เพื่อที่จะเอาไว้สำหรับ interpolate ฟังก์ชันใด ๆ ที่มี 2 ตัวแปร โอเคที่สำคัญคือ จะทำ linear interpolation ก่อนในทิศทางหนึ่ง และหลังจากนั้นก็จะทำการ linear interpolation ในทิศทางอื่น ๆ ต่อไป

สมมุติว่าเราต้องการที่จะหาค่าของฟังก์ชัน  $f$  ที่เราไม่เคารู้ ซึ่งอยู่ที่จุด  $P = (X,Y)$  เราจะสมมุติว่า เรารู้ค่าของ  $f$  ที่จุด 4 จุด ซึ่งได้แก่ จุด  $Q_{11} = (x_1,y_1)$  , จุด  $Q_{12} = (x_1,y_2)$  , จุด  $Q_{21} = (x_2,y_1)$  และจุด  $Q_{22} = (x_2,y_2)$  ดังรูปที่อยู่ด้านล่างนี้



รูปที่ 2.6

จุดสีแดงแสดงจุดแสดงจุดของข้อมูลที่เรารู้ค่า

จุดสีเขียวคือจุดที่เราต้องการที่จะ interpolate

อันดับแรกเราจะทำ linear interpolation ในทิศทางตามแกน x ซึ่งก็จะทำให้ได้สมการออกมาดังรูปข้างล่างนี้

$$f(R_1) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21}) \quad \text{where } R_1 = (x, y_1), \quad (2.1)$$

$$f(R_2) \approx \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22}) \quad \text{where } R_2 = (x, y_2). \quad (2.2)$$

เราจะทำต่อไปโดยการ interpolate ในทิศทางของแกน y ด้วยเช่นกัน ทำให้ได้

$$f(P) \approx \frac{y_2 - y}{y_2 - y_1} f(R_1) + \frac{y - y_1}{y_2 - y_1} f(R_2). \quad (2.3)$$

ในที่สุดเราจะได้ค่าของ  $f(x,y)$  ที่ต้องการโดยประมาณเป็นดังนี้

$$f(x, y) \approx \frac{f(Q_{11})}{(x_2 - x_1)(y_2 - y_1)} (x_2 - x)(y_2 - y) + \frac{f(Q_{21})}{(x_2 - x_1)(y_2 - y_1)} (x - x_1)(y_2 - y) \quad (2.4)$$

$$+ \frac{f(Q_{12})}{(x_2 - x_1)(y_2 - y_1)} (x_2 - x)(y - y_1) + \frac{f(Q_{22})}{(x_2 - x_1)(y_2 - y_1)} (x - x_1)(y - y_1). \quad (2.5)$$

ถ้าเราเลือกระบบ coordinate ที่ซึ่งจุด 4 จุดของ  $f$  นั้นถูกรู้ว่าเป็น  $(0,0)$ ,  $(0,1)$ ,  $(1,0)$ , และ  $(1,1)$  ดังนั้นสูตรในการ interpolation ก็จะทำให้อยู่ในรูปอย่างง่ายได้เป็น

$$f(x, y) \approx f(0, 0) (1 - x)(1 - y) + f(1, 0) x(1 - y) + f(0, 1) (1 - x)y + f(1, 1)xy. \quad (2.6)$$

หรือถ้าจะเขียนให้อยู่ในรูปของการกระทำทาง matrix ก็จะสามารถเขียนได้เป็น

$$f(x, y) \approx \begin{bmatrix} 1 - x & x \end{bmatrix} \begin{bmatrix} f(0, 0) & f(0, 1) \\ f(1, 0) & f(1, 1) \end{bmatrix} \begin{bmatrix} 1 - y \\ y \end{bmatrix} \quad (2.7)$$

ในทางตรงกันข้ามกับสิ่งที่เคยกล่าวมา การ interpolate นั้นจะไม่เป็น linear เพราะมันอยู่ในรูปด้านล่างนี้แทน

$$(a_1x + a_2)(a_3y + a_4), \quad (2.8)$$

ดังนั้นมันจะเป็นผลคูณของ linear function 2 อัน ในทางกลับกันเราสามารถที่จะเขียนการ interpolate ให้อยู่ในรูปดังข้างล่างนี้ได้

$$b_1 + b_2x + b_3y + b_4xy. \quad (2.9)$$

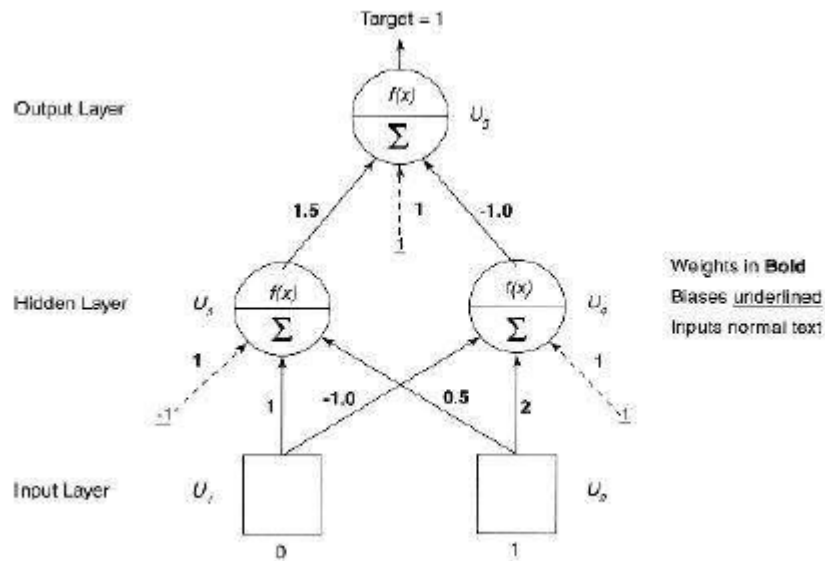
ในสองกรณีข้างต้น จำนวนของค่าคงที่ (ซึ่งมี 4 อัน) สัมพันธ์กับจำนวนของจุดข้อมูล เมื่อ  $f$  ถูกกำหนดค่าให้

ผลลัพธ์ของ bilinear interpolation จะเป็นอิสระจากลำดับของการ interpolate ถ้าหากว่าเราทำการ linear interpolation ในทิศทางของแกน  $y$  ก่อนอันดับแรก จากนั้นทำในทิศทางของแกน  $x$  ค่าประมาณของผลลัพธ์ที่ได้ก็จะมีค่าที่เท่ากัน

## 2.2 ทฤษฎีทางด้าน Neural network

สำหรับในส่วนของการ recognition สำหรับตัว ocr นั้น จากการที่ได้ไปศึกษา paper ต่างๆมานั้น ส่วนใหญ่แล้วก็จะใช้วิธีการที่เรียกกันว่า Back-propagation หรือการเรียนรู้แบบโครงข่ายประสาทเทียมแบบแพร่กระจายย้อนกลับ สำหรับตัว neural network ที่ข้าพเจ้าได้สร้างมานั้นจะเป็น neural network แบบที่มีหลายๆชั้นหรือที่เรียกกันว่า Multiple-Layer Perceptrons (MLP) โดยปกติแล้วตัว MLP เองก็จะประกอบไปด้วย layer ที่สำคัญ 3 layer คือ input layer , hidden layer , output layer ส่วนใหญ่เท่าที่พบมานั้น MLP ก็จะมีอยู่ 3 layer ก็เพียงพอที่จะใช้แก้ปัญหาได้เป็นจำนวนมาก แต่บางที MLP อาจจะมีถึง 4 layer ก็เป็นไปได้เช่นกัน รูปข้างล่างนี้แสดงถึง multiple layer network





รูปที่ 2.7 โครงสร้างอย่างง่ายของ MLP

สำหรับวิธีการของ back-propagation จะเป็นดังนี้คือ

อัลกอริทึมจะเริ่มต้นโดยการกำหนด weight ซึ่งเป็นเส้นเชื่อมระหว่าง node ต่างๆที่อยู่ในแต่ละ layer เข้าด้วยกัน โดยเราจะกำหนดค่าให้กับแต่ละ weight แบบ random ด้วย หลังจากที่เรากำหนดค่าแบบ random ให้กับ weight แล้วกระบวนการที่จะกล่าวต่อไปนี้จะมีการทำซ้ำจนกระทั่งค่าของ mean-squared error (MSE) ของ output layer มีค่าน้อยเพียงพอ

ในการที่จะใช้ neural network แบบ backpropagation นั้น เราจะต้องทำให้มันเกิดการเรียนรู้ก่อน โดยเราจะต้องทำการ train neural network เพื่อให้มันสามารถที่จะใช้ในการ recognition อะไรก็ตามที่เหมือนหรือคล้ายๆกับ ตัวอย่างที่เอาไป train ได้ ดังนั้นสิ่งที่เราจะต้องมีสำหรับ neural network แบบนี้ก็คือ training data

0.ทำการกำหนด weight ให้กับเส้นเชื่อมที่เชื่อม node ต่าง ๆ อย่าง random ก่อน

1.ใส่ data ตัวอย่างที่เราต้องการจะ train ให้กับ input layer และ output ที่ถูกต้องของ data ตัวอย่างให้กับ output layer

2.คำนวณการแพร่กระจายไปข้างหน้าของ data ตัวอย่างผ่าน network (คำนวณค่าผลรวมของ weight ของเส้นเชื่อมใน network,  $S_i$ , และค่าที่ได้จากฟังก์ชัน activation,  $u_i$ , สำหรับทุก ๆ เซลล์)

3.เริ่มต้นที่ output ให้มองย้อนผ่านทาง output และ เซลล์ที่อยู่ตรงกลาง คำนวณค่าของ errorตามสมการข้างล่างนี้ด้วย

$$\begin{aligned}\delta_o &= (C_i - u_5)u_5(1 - u_5) && \text{For the output cell} \\ \delta_i &= \left( \sum_{m:m>i} w_{m,j} \delta_o \right) u_i(1 - u_i) && \text{For all hidden cells}\end{aligned}\tag{2.10}$$

(สังเกตว่า m จะหมายถึงเซลล์ทั้งหมดที่ถูกเชื่อมไปกับ hidden node w คือค่า weight ที่กำหนดให้ และ u คือ ค่าที่ได้จาก activation function)

4. ในท้ายที่สุด weight ที่อยู่ภายใน network จะถูก update ใหม่ตามสมการข้างล่างนี้

$$\begin{aligned}w^*_{i,j} &= w_{i,j} + \rho \delta_o u_i && \text{For weights connecting hidden to output} \\ w^*_{i,j} &= w_{i,j} + \rho \delta_i u_i && \text{For weights connecting hidden to output}\end{aligned}\tag{2.11}$$

เมื่อ  $\rho$  หมายถึง learning rate (หรือขนาดของการก้าว) ค่าเล็ก ๆ นี้ จะเป็นตัวจำกัดการเปลี่ยนแปลงที่อาจจะเกิดขึ้นระหว่างแต่ละ step ค่าของ  $\rho$  สามารถถูกจูนเพื่อจะใช้ในการตัดสินใจว่าอัลกอริทึมแบบ back-propagation จะเข้าถึงจุดที่ต้องการที่เป็นคำตอบได้เร็วเพียงใด เราควรที่จะกำหนดค่าเริ่มต้นให้กับ  $\rho$  ให้เป็นค่าน้อย ๆ (0.1) เพื่อเป็นการทดสอบจากนั้นก็ค่อย ๆ เพิ่มค่าขึ้น

สำหรับการหาค่าของ learning rate นั้นมี trick เล็กน้อย คือ ถ้าหากว่าเรา กำหนดค่าของ learning rate ที่มากเกินไปก็จะทำให้กระบวนการของ neural network นั้นรวดเร็ว แต่ว่าค่าที่มากเกินไปเองอาจจะทำให้เกิดการออสซิลเลตได้เนื่องจากเมื่อใกล้ เข้าสู่ solution มันอาจจะเลยไปได้จึงต้องย้อนกลับไปมาหลายรอบ แต่ถ้าหากว่าเรากำหนดค่าของ learning rate ที่น้อยเกินไปก็อาจทำให้กระบวนการของ neural network นั้นช้าเกินไปกว่าที่เราจะหาคำตอบได้ ดังนั้นทางที่ดีควรจะมีการทำ neural network ที่สามารถที่จะปรับค่าของ learning rate ในขณะที่กำลัง train อยู่ได้